

Assistente Médico

Tech Challenge – Fase 3

Jefferson Antônio Pantoja Silva | 12 de setembro de 2026

O desafio

Como responder dúvidas médicas com contexto, rastreabilidade e segurança?

O sistema deve combinar conhecimento biomédico, informações do paciente e mecanismos explícitos de proteção.

Objetivo

Construir um pipeline modular que prepare dados, ajuste uma LLM, consulte prontuários sintéticos e gere respostas contextualizadas.

Limite de uso

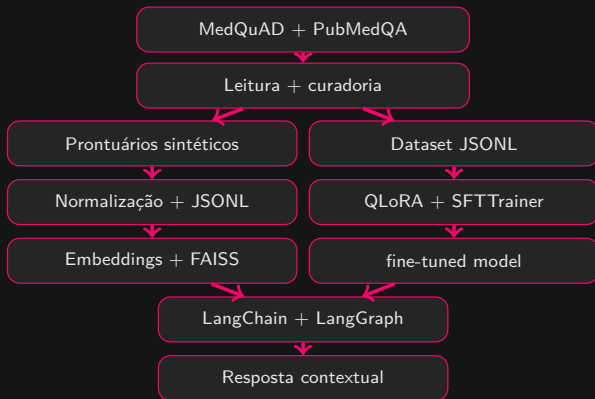
Apoio informacional. Não substitui avaliação ou decisão de um profissional de saúde.

Visão geral da solução

Ideia central

O modelo treinado com o dataset gera a resposta com base nas informações dos prontuários sintéticos criados a partir do dataset

O pipeline separa preparação, conhecimento do modelo e contexto do paciente.



Dados: curadoria antes do modelo

Fontes públicas

- ▶ MedQuAD: XML com perguntas e respostas do NIH.
- ▶ PubMedQA: JSON com pergunta, contexto e resposta biomédica.

Curadoria

Normalização, validação, remoção de vazios e duplicidades. O campo source é preservado para rastreabilidade e o JSONL é escrito incrementalmente.

Saída: finetuning_qa.jsonl

```
{ "source": "MedQuAD:arquivo:pid",  
  "text": "ANSWER THE QUESTION."  
[|Context|] ...[|eContext|]  
[|Question|] ...[|eQuestion|]  
[|Answer|] ...[|eAnswer|]"}
```

Mesmo template na inferência: reduz o desalinhamento entre treino e uso do modelo.

Fine-tuning no Colab

1. Dataset separado em treino e teste.
2. Modelo base Llama-3.2-1B-Instruct em 4-bit.
3. Adaptação com QLoRA e Unsloth.
4. Treino supervisionado com `trl.SFTTrainer`.
5. Merge do adapter e salvamento do modelo final.

O treinamento é executado no Google Colab; a inferência usa o modelo local mesclado.

Parâmetros principais

LoRA $r = 16$ | $\alpha = 32$
dropout = 0,05
batch = 2 | acumulação = 8
learning rate = 2×10^{-4}
1 época | BF16/FP16

Prontuários sintéticos e rastreabilidade

Um caso curado é convertido em um prontuário fictício com estrutura consistente:

- ▶ queixa principal e histórico;
- ▶ medicamentos e sinais vitais;
- ▶ avaliação e plano;
- ▶ origem e identificador do paciente.
- ▶ OpenAI API (gpt-4o-mini)

Confiabilidade do pipeline

Os lotes são validados pela presença de exatamente uma ocorrência de cada source. Respostas inválidas podem ser reprocessadas até três vezes.

Retomada

O JSONL é gravado progressivamente e checkpoints evitam repetir registros já concluídos.

Avaliação do modelo fine-tuned

Protocolo

- ▶ Amostra aleatória de 500 itens do teste.
- ▶ Teste separado via `train_test_split`.
- ▶ Apenas o fine-tuned; sem comparação com o modelo base.

Interpretação

Métricas textuais não comprovam factualidade ou validade clínica.

Resultados registrados

Métrica	Resultado
Exact Match	0,0000
Token F1	0,3280
ROUGE-L	0,2717
BERTScore F1	0,8354

Exact Match nulo e Token F1/ROUGE-L moderados indicam baixa correspondência literal. O BERTScore sugere similaridade semântica, mas não garante correção factual.

Contexto clínico com controle de risco

Recuperação

- ▶ Embeddings com all-MiniLM-L6-v2 e busca vetorial com FAISS.
- ▶ Até quatro documentos relevantes.
- ▶ Filtro opcional pelo `patient_id`.
- ▶ Fontes retornam junto da resposta.

Salvaguardas

- ▶ O campo `plan` fica fora do contexto RAG.
- ▶ Pedidos de medicamento ou tratamento são revisados.
- ▶ A resposta inclui aviso de avaliação profissional.
- ▶ Há fallback se a revisão externa falhar.

Arquitetura e fluxo RAG

Clean Architecture

- ▶ domain: modelos imutáveis.
- ▶ application: casos de uso e portas.
- ▶ infrastructure: LLM, FAISS, OpenAI e Langfuse.
- ▶ presentation: controladores CLI.

translate_question: OpenAI API

retrieve: embeddings + FAISS

build_context: registros relevantes

generate: LLM local fine-tuned

translate_answer: OpenAI API

review_output: OpenAI API + fontes

Demonstração!